CHAPITRE 3

Structure conditionnelle

Plan du chapitre

I Booléens **2**
- A - Définitions et exemple 2
- B - Opérations 3

II Instructions `if` et `if/else` **4**
- A - `if` 4
- B - `if...else` 5
- C - Le rôle de l'indentation 6
- D - Instruction `if... elif` 7

III Exercices **10**

Partie I Booléens

A - Définitions et exemple

Le test `if` ("si") permet de demander à Python de n'éxécuter une partie du code que si une certaine condition est vérifiée. Cela s'avère très utile dans des situations où plusieurs alternatives se dégagent, mais aussi pour gérer des exceptions.

Exemple :

On note en pseudo-code un code de structure conditionnelle

PseudoCode
```
Heure←7
Si Heure > 8 :
    Afficher("Les cours ont commencé !")
Sinon :
    Afficher("Il est trop tôt pour être au lycée !")
```

Que se passe-t-il après l'exécution de ce script ?

.....

La ou les conditions sous lesquelles une partie du code va être exécutée s'expriment par des **booléens**.
Il s'agit d'un type de variable Python qui ne peut prendre que deux valeurs : `True` et `False`.

Exemple :

Imaginons qu'on dispose d'une variable `nb_eleve` représentant le nombre d'élèves dans une classe et que cette variable ait été initialisée à 30.
On construit des booléens grâce aux opérateurs suivants :

Opérateur	Description	Exemple
`==`	est égal à	`nb_eleve == 30` renvoie .....
`!=`	est différent de	`nb_eleve != 30` renvoie .....
`>=`	est supérieur (ou égal) à	`nb_eleve >= 28` renvoie .....
`>`	est supérieur strict à	`nb_eleve > 32` renvoie .....
`<=`	est inférieur (ou égal) à	`nb_eleve <= 31` renvoie .....
`<`	est inférieur strict à	`nb_eleve < 30` renvoie .....

Erreur fréquente : Ne pas confondre
Il ne faudra pas confondre le symbole = de l'affectation, et le symbole == qui permet de tester si deux valeurs sont égales. L'expression 1 = 2 renvoie un message d'erreur (on ne pas affecter la valeur 2 dans 1), tandis que 1 == 2 renvoie le booléen .....

Ex 1 ★☆☆☆ **Booléens** .....

Compléter la console Python suivante :

console ×

```
>>> 1==0
...
>>> 1!=0
...
>>> 5>4
...
>>>-5>-4
...
>>>2**3==6
...
>>>6%2==3
...
```

B - Opérations

On peut, même combiner les booléens entre eux à l'aide de connecteurs logiques :

Connecteur	Description	Exemple
`and`	ET	`1==1 and 1==2` renvoie .....
`or`	OU (inclusif)	`1==1 or 1==2` renvoie .....
`not`	NON (négation)	`not 1==1` renvoie .....

Ex 2 ★★☆☆ **Opérations** .....

On suppose qu'une variable réelle x a été définie préalablement.
Construire les booléens exprimant les assertions suivantes :

1. $x \in [2, +\infty[$

   .....

2. $x \in [2, 3[$

   .....

3. $x \in]-\infty, 1[\cup [2, +\infty[$

   .....

Ex 3 ★★☆☆ .....

Soient `a` et `b` des variables entières.
Donner un booléen exprimant les conditions suivantes :

- `a` vaut 3 et `b` ne vaut pas 5 ;
- `a` est supérieur à `b` ou inférieur à `-b` ;
- `a` et `b` sont de même signe.

.....

.....

Ex 4 ★★★☆ **Multiples** .....

Soient `a` et `b` des variables entières.
Donner un booléen exprimant les conditions suivantes :

- a et b sont impairs ;
- a est un multiple de 3 ;
- b n'est pas un multiple de 3 ;
- a et b ont le même chiffre des unités (en base 10).

.....

.....

Partie II Instructions `if` et `if/else`

A - `if`

testif.py

```
if condition :
    # Vos instructions si la condition est satisfaite
# Vos instructions en dehors du test
```

Si la `condition` est vérifiée, Python "rentre dans `if`" et exécute les instructions. Si elle ne l'est pas, Python ne fait rien et passe directement aux instructions en dehors du test.

Exemple :

La script suivant permet d'afficher si l'utilisateur est majeur ou non.

majorite.py

```
age=input("Entrer votre age")
if age<18 :
    print("Mineur")
```

Quelques remarques :

- Les deux points à la fin de la condition sont indispensables.
- La condition booléenne `condition` doit être écrite sur une seule ligne.
- On peut faire suivre plusieurs tests à la suite, ou imbriqués les uns dans les autres.

Ex 5 ★☆☆☆ .....

Compléter la fonction suivante pour qu'elle renvoie x^2 si $x \geq 1$ et 0 sinon. Et compléter la console.

tpexo.py

```
#Script Python
def ma_fonction(x) :
    if          :
        y=
    if          :
        y=
    return y
```

console

```
Python 3.11.6 /usr/bin/python
>>> #Run tpexo.py
>>> ma_fonction(1)
...
```

Ex 6 ★★☆☆ **Résultat du Bac** .....

Écrire une fonction Python qui prend en argument la moyenne générale à l'examen d'un candidat au Bac et renvoie son statut : admis, rattrapage, recalé
Puis écrire une commande qui permet de renvoyer le résultat pour un élève ayant eu 11.5 de moyenne générale au Bac.

tpexo.py ×

```
#Script Python
.
.
.
.
.
.
.
```

console ×

```
Python 3.11.6 /usr/bin/python
>>> #Run tpexo.py
>>>
...
```

On peut utiliser un test `if` seul, mais bien souvent on veut traiter une alternative de deux cas :

Si telle condition est vérifiée alors faire..., sinon faire..."

Pour alléger ce cas de figure, il existe une commande `else` ("sinon").

B - if...else

La syntaxe d'une alternative de cas faite à l'aide d'un `else` est la suivante :

testifelse.py ×

```
if condition :
    # Vos instructions si la condition est satisfaite
else :
    # Vos instructions si la condition n'est pas satisfaite
# Vos instructions en dehors du test
```

Si la `condition` est vérifiée, Python exécute les instructions du bloc `if` puis ignore celles du bloc `else`. Inversement si la condition 'est pas satisfaite, Python ignore les instructions du bloc `if` et exécute celle du bloc `else`.

Exemple :

Complétons notre programme précédent :

majorite.py ×

```
age=input("Entrer votre age")
if age<18 :
    print("Mineur")
.
.
```

Quelques remarques :

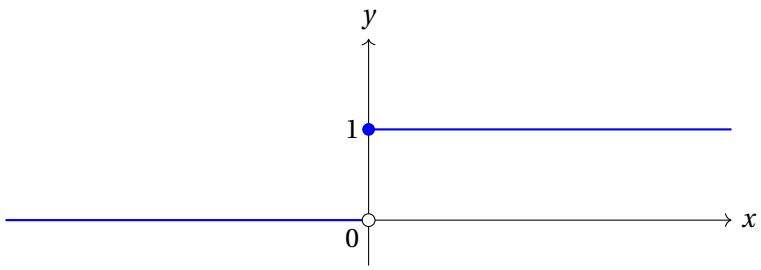
- La condition booléenne `condition` ne s'écrit qu'une seule fois : après le `if` et **pas après le** `else`.
- Le bloc `else` est au même niveau d'indentation que le bloc `if`.
- Les deux points et l'indentation sont toujours indispensables.

On fait le bilan des deux structures :

Structure	En Python
Si... Alors...	`if` *condition* : *Instructions à exécuter quand la condition est vraie* *Instructions en dehors du test*
Si... Alors... **Sinon...**	`if` *condition* : *Instruction 1 à exécuter quand la condition est vraie* `else` : *Instruction 2 à exécuter quand la condition est fausse* *Instructions en dehors des tests*

Ex 7 ★★☆☆ **Fonction rampe** .....

On appelle fonction rampe la fonction qui vaut 0 sur l'intervalle $]-\infty, 0[$ et 1 sur l'intervalle $[0, +\infty[$.



Écrire la fonction Python déclarant cette fonction rampe
Et tester la sur votre ordinateur pour vérifier quelle fonctionne.

.....
.....
.....
.....
.....
.....
.....

C - Le rôle de l'indentation

En Python, l'indentation automatiques des *instructions* est très importante. L'annulation de l'indentation marque la sortie d'une structure conditionnelle.
Voyons cela sur plusieurs exemples :

Exemple :

- **Premier exemple :**

affichage.py ×

```
if p>50:
    p=p-10
    print(p)
```

L'instruction `print(p)` est exécutée uniquement si .....
L'affichage ne s'effectuera pas dans tous les cas.

- **Deuxième exemple :**

affichage.py ×

```
if p>50:
    p=p-10
print(p)
```

L'instruction `print(p)` est exécutée pour .....
L'affichage s'effectuera dans tous les cas.

- **Troisième exemple :**

affichage.py ×

```
if p>50:
    p=p-10
else :
    p=p-5
print(p)
```

Que se passe-t-il si p a comme valeur :

- 30 ? .....
- 70 ? .....

L'instruction `print(p)` est exécutée pour .....
L'affichage s'effectuera dans tous les cas.

D - Instruction `if... elif`

Si l'on souhaite considérer une situation à plus de deux alternatives, on peut utiliser la commande `elif`. Il s'agit de la contraction de `else` et `if` ("sinon, si").
Il s'agit donc de donner des instructions dans le cas où **une première condition n'est pas vérfiée et une deuxième condition est vérifiée**.
En voici la syntaxe :

testelif.py ×

```
if condition1 :
    # Vos instructions si la condition1 est satisfaite
elif condition2 :
    # Vos instructions si la condition1 n'est pas satisfaite
        mais que la condition 2 l'est.
# Vos instructions en dehors du test
```

Si la condition booléenne `condition1` est satisfaite, Python exécute le premier bloc d'instructions et ignore le deuxième (**attention**, même si `condition2` est satisfaite).
Si la condition booléenne `condition1` n'est pas satisfaite, Python ignore le premier bloc d'instructions, et n'exécute le deuxième que si `condition2` est satisfaite.

Exemple :

Considérons deux scripts

elif.py ×

```
a=1
if a>2 :
    a=a+1
elif a>0 :
    a=a-4
```

La valeur de a après exécution est : ....

elif.py ×

```
a=0
if a<2 :
    a=a+1
elif a<3 :
    a=a-4
```

La valeur de a après exécution est : ....

On peut enchaîner les commandes `elif`. Dans ce cas, tout nouveau bloc d'instructions suivant une commande `elif condition_N` ne sera traité qu'à condition : qu'aucune des conditions précédentes n'ait été satisfaites et que cette fameuse : `elif condition_N` soit satisfaite.
Concrètement on peut enchaîner autant de commandes `elif` mais l'instructions suivant la commande `elif condition100` ne sera traité uniquement si les 99 instructions précédentes n'ont pas été satisfaites et que la `condition100` est satisfaite.
Pour finir, il est également possible de conclure une distinction de cas par une commande `else` donnant des instructions dans le cas où aucune condition testée précédemment n'ait été satisfaite.

Exemple :

Considérons le script suivant :

elif.py ×

```
temperature = -2
if temperature > 30:
    print("Il fait chaud")
elif temperature >= 0:
    print("Il ne gele pas")
else:
    print("Il gele")
```

Après exécution de notre script il s'affiche ..... en effet le `else` est le complémentaire de l'union des deux autres conditions.

Dans le cas où l'on enchaîne des commandes `elif`, il faut faire particulièrement attention à la façon dont les conditions demandées s'excluent les unes par rapport aux autres. En particulier l'ordre des conditions a son importance.

Exemple :

Voyons cette dernière remarque sur un exemple :

elif.py

```
temperature = 33
if temperature > 30:
    print("Il fait chaud")
elif temperature >= 0:
    print("Il ne gele pas")
else:
    print("Il gele")
```

Ce premier script affiche : .....
Alors que le script ci-dessous :

elif.py

```
temperature = 33
if temperature >= 0:
    print("Il ne gele pas")
elif temperature > 30 :
    print("Il fait chaud")
else:
    print("Il gele")
```

affichera plutôt .....
En effet la condition booléenne ..... est moins restrictive que la condition booléenne .....

Ex 8 ★★☆☆ .....

Compléter la console suivante :

tpexo.py

```
#Script Python
def fun (a) :
    if  a>=10 :
        return a+1
    elif a>=5 :
        return a+1
    elif a>=0 :
        return 2*a
    else :
        return 3*a
```

console

```
Python 3.11.6 /usr/bin/python
>>> #Run tpexo.py
>>> fun(12)
...
>>> fun(7)
...
>>> fun(3)
...
>>> fun(-1)
...
```

Partie III Exercices

Ex 9 ★☆☆☆ Prédiction

Étudier la portion de code suivante, prévoir le résultat de son exécution et tester la dans la console :

console ×

```
Python 3.11.6 /usr/bin/python
>>> x=10
>>> x<0 or x>=5
...
```

Ex 10 ★★☆☆

Écrire une fonction qui prend en argument un nombre réel `x` et qui renvoie les booléens `True` ou `False` selon que `x` appartienne à l'ensemble :

$$[1,2]\cup]3,7]\cup[28,+\infty[$$

Vérifier votre fonction sur l'ordinateur avec différentes valeurs de `x`.

Ex 11 ★★☆☆ Pair, impair

1. Écrire une fonction Python `fun1` prenant en argument un entier `n` et renvoyant la chaîne de caractères `"pair"` si `n` est pair, et la chaîne `"impair"` sinon.
   On proposera trois écritures.
2. Écrire maintenant une fonction Python `fun2` prenant en argument un entier `n` et renvoyant le booléen `True` si `n` est pair et `False` si `n` est impair.

.....

.....

.....

Ex 12 ★★☆☆ **Valeur absolue** .....

Écrire la fonction valeur absolue en Python :

.....

.....

.....

.....

.....

.....

Ex 13 ★★★★ **Bissextile ?** .....

1. Écrire une fonction `est_bissextile` qui prend comme argument une année et qui renvoie le booléen `True` si l'année est bissextile et `False` sinon.
   Une année bissextile si elle vérifie l'une des deux conditions :
   - *c'est un multiple de 4 et pas une multiple de 100 ;*
   - *c'est un multiple de 400.*

   Par exemple 2000,2004 et 2008 sont des années bissextiles mais 1900,2100 et 2200 ne le sont pas.

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

2. À l'aide de la fonction précédente, écrire une fonction qui prend en argument un mois de l'année et l'année courante, et qui renvoie le nombre d'heures dans le mois.
   On rappelle que durant les années bissextiles, le mois de février a 29 jours, sinon il en a 28.

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

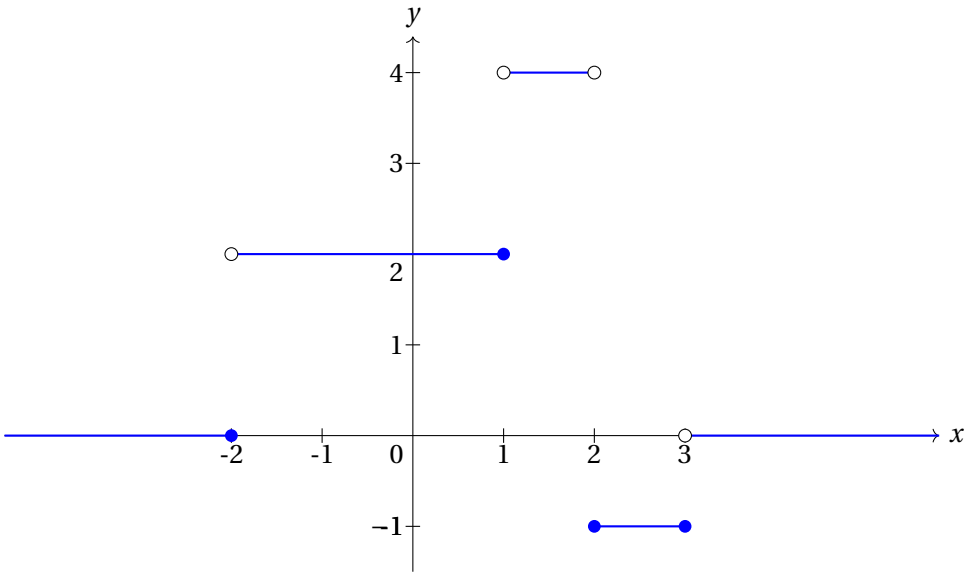
Ex 14 ★★★☆ **Fonction par morceaux** .....

1. Écrire une fonction Python `fun3` qui prend en argument une variable réelle `x` et qui renvoie la valeur de $f(\mathtt{x})$ où f est la fonction définie par morceaux suivante :

$$f(x) = \begin{cases} x+1 & \text{si } x \leq 0 \\ 1 & \text{si } 0 < x \leq 2 \\ x-1 & \text{si } 2 < x \end{cases}$$

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

2. Écrire une fonction Python `fun4` prenant en argument une variable réelle `x` et renvoyant la valeur de $g(\mathtt{x})$ où g est la fonction "en escalier" dont le graphe est le suivant :



.....
.....
.....
.....
.....
.....
.....
.....

Ex 15 ★ **Formule de Héron** .....

La formule de Héron, du nom du mathématicien grec Héron d'Alexandrie, permet d'exprimer l'aire d'un triangle à partir de la longueur de ses côtés uniquement (sans avoir besoin de déterminer sa hauteur). Si l'on dispose d'un triangle dont les côtés ont pour longueur a, b et c, alors la **formule de Héron** indique que son aire vaut :

$$\sqrt{\left(\frac{a+b+c}{2}\right)\left(\frac{a+b+c}{2}-a\right)\left(\frac{a+b+c}{2}-b\right)\left(\frac{a+b+c}{2}-c\right)}$$

1. Écrire une fonction `aire` prenant en argument trois nombres réels `a,b,c` et renvoyant l'aire du triangle dont les côtés ont pour longueur `a,b` et `c`.
   Testez cette fonction sur le triangle rectangle donc les côtés ont pour longueur 3, 4 et 5.
   D'ailleurs pourquoi ce triangle est-il rectangle ?

   .....
   .....
   .....
   .....
   .....
   .....
   .....
   .....
   .....
   .....

2. Avez-vous recopié 4 fois la fraction $\frac{a+b+c}{2}$? Si oui, peut-on coder de manière plus économique ?

   .....
   .....
   .....
   .....
   .....
   .....
   .....
   .....

La formule de Héron n'est valable que si la quantité sous la racine carrée est positive.
On peut montrer que c'est le cas si, et seulement si, les trois nombres `a,b,c` vérifient les inégalités triangulaires :

$$a+b \geq c, \quad a+c \geq b \quad \text{et} \quad b+c \geq a$$

3. Écrire une fonction Python `admissible` prenant en arguments trois nombres réels `a,b,c` et qui vérifie si ces trois réels correspondent aux trois longueurs des côtés d'un triangle.
   Tester cette fonction pour différents triplets de réels.

   .....
   .....
   .....
   .....